

Videogame Semantic Search

Un'ontologia OWL 2 DL per il dominio dei videogiochi
Modellazione, reasoning e interrogazione

Videogame Semantic Search Project

Agosto 2026

Cosa abbiamo costruito

Un'ontologia **OWL 2 DL** sul dominio dei videogiochi (2010–2026), popolata con dati estratti da Wikidata e interrogabile tramite SPARQL.

- **Perché:** rappresentare in modo formale e ragionabile la conoscenza su giochi, studi, piattaforme, generi, franchise e premi
- **Cosa contiene:**
 - 78 classi (15 definite) e 57 proprietà con caratteristiche OWL 2
 - 12 blocchi di disjointness, 3 property chain, 3 chiavi (hasKey)
 - Allineamento a Wikidata e Dublin Core
- **Come si usa:** reasoning a due livelli (OWL 2 RL + SPARQL CONSTRUCT) che classifica automaticamente le istanze; un'applicazione web (NL → SPARQL, grafo interattivo) la rende esplorabile

- **Dominio:** videogiochi 2010–2026 con sviluppatori, publisher, generi, piattaforme, personaggi, franchise, premi, motori, modalità, recensioni ed eventi
- **Namespace:** <http://www.videogame-ontology.org/ontology#> (prefisso vg:)
- **Schema e istanze separati:** videogames.owl (schema, 934 triple) e videogames_pruned_2020.owl (istanze demo, ~745k triple)

Componente	Qtà	Componente	Qtà
Classi totali	78	Classi definite (equivalentClass)	15
Classi primitive	61	Classi con vincolo di cardinalità	2
Object property	31	Blocchi AllDisjointClasses	12
Datatype property	26	propertyChainAxiom	3
Coppie inverse	12	hasKey	3
Proprietà transitive	3	Allineamenti (Wikidata + Dublin Core)	4 + 3

12 classi top-level (disgiunte)

- VideoGame
- Developer
- Publisher
- Genre
- Platform
- Character
- Franchise
- Award
- GameEngine
- GameMode
- Review
- GameEvent

Platform — partizionamento a due livelli

- **ConsolePlatform**: PlayStation, Xbox, Nintendo
- **PCPlatform**: Windows, MacOS, Linux
- **MobilePlatform**: iOS, Android

Altre sottoclassi: 10 generi (aperti), stato di rilascio, budget (Indie/AAA), tipologie di premio, personaggio e motore

Pattern modellistici

Review — relazione N-aria: gioco (reviewedGame) + revisore + punteggio + data

GameEvent — ciclo di vita: Announcement, Release, Update, Delisting

GameMode — value partition: 4 modalità disgiunte

Caratteristica	#	Esempi
<code>owl:inverseOf</code>	12 coppie	<code>developedBy/developerOf</code> , <code>belongsTo/includes</code> , ...
<code>TransitiveProperty</code>	3	<code>sequelOf</code> , <code>partOfFranchise</code> , <code>subsidiaryOf</code>
<code>SymmetricProperty</code>	3	le super-proprietà delle 3 catene (<code>sharesDeveloperWith</code> , ...)
<code>Asymmetric / Irreflexive</code>	13 / 18	<code>developedBy</code> , <code>hasGenre</code> , <code>belongsTo</code> , ...
<code>FunctionalProperty</code>	14	<code>reviewedGame</code> + 13 datatype (<code>releaseDate</code> , <code>metacriticScore</code> , ...)
<code>subPropertyOf</code>	9	<code>gameName</code> , <code>developerName</code> , ... $\sqsubseteq$ name

Vincolo globale di OWL 2 DL

Le transitive e le loro inverse non sono *semplici*: non possono essere asimmetriche, irriflessive, funzionali, né comparire in cardinalità o `hasKey`.

3 Property Chain Axiom (inclusione di ruoli)

$\text{belongsTo} \circ \text{includes} \sqsubseteq \text{sharedFranchiseWith}$
 $\text{developedBy} \circ \text{developerOf} \sqsubseteq \text{sharesDeveloperWith}$
 $\text{publishedBy} \circ \text{publisherOf} \sqsubseteq \text{sharesPublisherWith}$

- Composizione ben tipata: $\text{VideoGame} \xrightarrow{\text{belongsTo}} \text{Franchise} \xrightarrow{\text{includes}} \text{VideoGame}$
- Le super-proprietà sono Symmetric; la catena include anche la coppia (g, g) , per questo nelle query si filtra $?g \neq ?\text{altro}$

3 chiavi owl:hasKey

$\text{VideoGame}(\text{hasWikidataId}) \cdot \text{VideoGame}(\text{hasSteamAppId}) \cdot \text{Review}(\text{reviewedGame}, \text{reviewerName})$

Identità espressa con chiavi (valide sui soli individui nominati) anziché con `InverseFunctionalProperty` su datatype, non ammessa in OWL 2 DL.

Ontologia — Classi Definite (Condizioni Necessarie e Sufficienti)

Classe	Definizione (owl:equivalentClass)
AwardWinningGame	<code>VideoGame $\sqcap$ $\exists$ wonAward.Award</code>
FranchiseGame	<code>VideoGame $\sqcap$ $\exists$ belongsTo.Franchise</code>
SequelGame	<code>VideoGame $\sqcap$ $\exists$ sequelOf.VideoGame</code>
Multiplayer / SinglePlayer / CoopGame	<code>VideoGame $\sqcap$ $\exists$ hasGameMode.<i>M</i></code>
PC / Console / MobileGame	<code>VideoGame $\sqcap$ $\exists$ availableOn.<i>P</i></code>
PlayStation / Xbox / NintendoGame	<code>VideoGame $\sqcap$ $\exists$ availableOn.<i>P</i></code>

M = la modalità corrispondente (es. `MultiplayerMode`); *P* = la classe di piattaforma corrispondente (es. `PCPlatform`, `PlayStationPlatform`)

- 12 classi definite con `intersectionOf + someValuesFrom` su object property
- Altre 3 (`HighlyRated`, `Recent`, `Standalone`) usano costrutti più espressivi → 15 in totale
- La gerarchia delle piattaforme propaga la classificazione (`PS5` ⇒ `PlayStationGame` e `ConsoleGame`)

Classe	Assioma	Costrutto
StandaloneGame	$\equiv \text{VideoGame} \sqcap \neg \exists \text{sequelOf}.\text{VideoGame}$	complementOf
HighlyRatedGame	$\equiv \text{VideoGame} \sqcap \exists \text{metacriticScore}.\text{integer}[\geq 85]$	minInclusive
RecentGame	$\equiv \text{VideoGame} \sqcap \exists \text{releaseDate}.\text{date}[\geq 2020-01-01]$	minInclusive
GameWithSingleDeveloper	$\sqsubseteq \text{VideoGame} \sqcap \leq 1 \text{ developedBy}$	maxCardinality
ExclusiveRelease	$\sqsubseteq \text{VideoGame} \sqcap \leq 1 \text{ availableOn}$	maxCardinality

Le cardinalità sono *non qualificate* (nessun owl:onClass).

- Le ultime due hanno **sole condizioni necessarie**: per l'Open World Assumption un reasoner dedurrebbe ≤ 1 solo con assiomi di chiusura, quindi fungono da vincoli di integrità e non da classificatori
- complementOf e le datatype restriction **escono dal profilo OWL 2 RL**: motivano il secondo livello di reasoning

```
# Classe definita con datatype restriction
vg:HighlyRatedGame a owl:Class ;
  owl:equivalentClass [
    owl:intersectionOf ( vg:VideoGame
      [ a owl:Restriction ;
        owl:onProperty vg:metacriticScore ;
        owl:someValuesFrom [
          a rdfs:Datatype ;
          owl:onDatatype xsd:integer ;
          owl:withRestrictions
            ( [ xsd:minInclusive 85 ] ) ] ] )
    ] .
```

```
# Property chain + simmetria
vg:sharesDeveloperWith
  a owl:ObjectProperty ,
    owl:SymmetricProperty ;
  owl:propertyChainAxiom
    ( vg:developedBy vg:developerOf ) .

# Condizione solo necessaria
vg:GameWithSingleDeveloper
  rdfs:subClassOf vg:VideoGame ,
    [ a owl:Restriction ;
      owl:onProperty vg:developedBy ;
      owl:maxCardinality
        "1"^^xsd:nonNegativeInteger ] .
```

12 blocchi owl:AllDisjointClasses: nessuna istanza può appartenere a due classi dello stesso blocco

#	Classi disgiunte
12	le classi top-level
3	Console / PC / MobilePlatform
3	PlayStation / Xbox / NintendoPlatform
3	Windows / MacOS / LinuxPlatform
2	IOS / AndroidPlatform
3	Industry / Editorial / UserAward

#	Classi disgiunte
2	Player / NonPlayerCharacter
2	Proprietary / OpenSourceEngine
4	Released / EarlyAccess / Cancelled / DelistedGame
2	IndieGame / AAAGame
4	SinglePlayer / Multiplayer / Coop / MMOMode
4	Announcement / Release / Update / DelistingEvent

- **Scelta deliberata:** SinglePlayerGame e MultiplayerGame *non* sono disgiunte (un gioco può avere entrambe le modalità); i generi restano una tassonomia aperta
- Con un reasoner DL le disjointness rilevano dati incoerenti provenienti da Wikidata

4 equivalentClass con Wikidata

`vg:VideoGame ≡ wd:Q7889`

`vg:Developer ≡ wd:Q210167`

`vg:GameEngine ≡ wd:Q193564`

`vg:Genre ≡ wd:Q65956376`

3 equivalentProperty con Dublin Core

`vg:releaseDate ≡ dcterms:issued`

`vg:gameName ≡ dcterms:title`

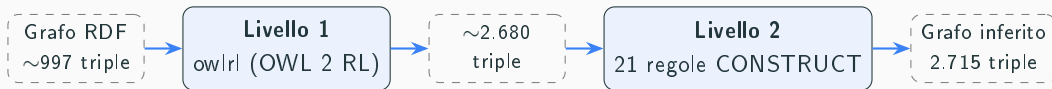
`vg:gameDescription ≡`

`dcterms:description`

Vincoli globali OWL 2 DL sulle proprietà

- Nessuna proprietà non semplice in asimmetria, irriflessività, funzionalità, cardinalità o `hasKey`
- Nessuna `InverseFunctionalProperty` su datatype property
- Verifica: **0 violazioni**

Nota: `xsd:date` non è nella datatype map di OWL 2 (solo `xsd:dateTime`); per usare `HermiT` va migrato a `xsd:dateTime`.



(valori sull'insieme di istanze di test: 3 giochi, 2 developer, piattaforme)

owlrl materializza

- `subClassOf`, `intersectionOf` + `someValuesFrom`
- `inverseOf`, `Symmetric`, `Transitive`
- `propertyChainAxiom`

Livello 2: regole CONSTRUCT

- datatype restriction: `FILTER(?s >= 85)`
- complementOf: `FILTER NOT EXISTS (NAF)`
- classificazioni e inverse (ridondanti con RL)

≤ 1 in superclasse è ammessa in RL (`cls-maxc2`) ma non classifica: serve solo come vincolo.

Asserito

```
# GameA = Elden Ring
vg:GameA a vg:VideoGame ;
  vg:releaseDate
    "2022-02-25"^^xsd:date ;
  vg:metacriticScore 96 ;
  vg:developedBy vg:Dev1 ;
  vg:hasGameMode vg:SinglePlayer ,
    vg:OnlineMultiplayer ;
  vg:availableOn vg:PS5 ,
    vg:PC_Windows ;
  vg:wonAward vg:TGAAward2022 .
vg:GameC vg:developedBy vg:Dev1 .
vg:PS5 a vg:PlayStationPlatform .
vg:PC_Windows a vg:WindowsPlatform .
```

Inferito

Tipi inferiti di GameA

PlayStation-, ConsoleGame
PCGame
Single-, MultiplayerGame
AwardWinningGame
HighlyRatedGame
RecentGame
StandaloneGame

Motivo

PS5 : PlayStationPlatform
Windows $\sqsubseteq$ PCPlatform
 $\exists$ hasGameMode.M
 $\exists$ wonAward.Award
 $96 \geq 85$
 $2022 \geq 2020$
nessun sequelOf

Relazioni inferite

Dev1 developerOf GameA inverseOf
GameA sharesDeveloperWith GameC catena + simm.

In tondo: livello 1 (owlrl). In corsivo: livello 2 (CONSTRUCT).

Classi inferite al posto dei filtri

```
PREFIX vg: <http://www.videogame-ontology.org/ontology#>
SELECT ?name ?score WHERE {
  ?g a vg:HighlyRatedGame ,
      vg:MultiplayerGame ,
      vg:PlayStationGame ;
      vg:gameName ?name ;
      vg:metacriticScore ?score .
}
ORDER BY DESC(?score) LIMIT 10
```

Nessun filtro su punteggio, modalità o gerarchia delle piattaforme: li ha già risolti il reasoner.

Relazioni derivate dalle catene

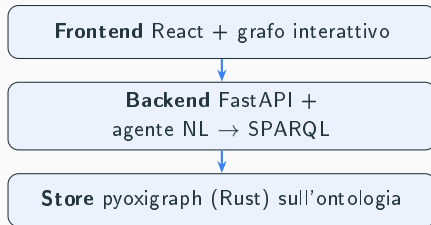
```
PREFIX vg: <http://www.videogame-ontology.org/ontology#>
SELECT DISTINCT ?other WHERE {
  ?er vg:gameName "Elden Ring" ;
      vg:sharesDeveloperWith ?g .
  ?g vg:gameName ?other .
  FILTER(?g != ?er)
}
```

sharesDeveloperWith non è mai asserita: deriva da developedBy o developerOf. Il FILTER esclude la coppia riflessiva prodotta dalla catena.

Popolamento da Wikidata

1. 12 tipologie di query SPARQL, ciascuna su 204 finestre anno $\times$ mese
2. Deduplicazione: 4.420 entità rimosse
3. Chiusura OWL 2 RL: $\sim 1\text{M} \rightarrow \sim 1.4\text{M}$ triple
4. Pruning: $\sim 1.17\text{M}$ triple (2010–2026)
5. Demo 2020–2026: $\sim 745\text{k}$ triple, $\sim 68.7\text{k}$ giochi

Architettura applicativa



pyoxigraph vs rdflib: query 5–62 $\times$ più veloci, caricamento ~ 2 s invece di ~ 41 s, RAM ridotta di circa 4 $\times$.

Grazie per l'attenzione

Videogame Semantic Search Project

Ontologia OWL 2 DL · Reasoning · Interrogazione SPARQL